

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом простых итераций (метод Якоби).

Студент гр. 4342

Ларионов В.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Разработать программу на языке C++ для численного решения системы линейных алгебраических уравнений (СЛАУ) методом Якоби. Исследовать сходимость метода на матрицах с диагональным преобладанием, сравнить полученное приближённое решение с точным, оценить влияние критерия останова и максимального числа итераций.

Теоретический минимум.

Метод Якоби – итерационный метод решения СЛАУ вида $Ax = b$. Матрица A представляется в виде суммы диагональной D и двух треугольных матриц L (нижняя) и U (верхняя): $A = D + L + U$.

Условие сходимости. Достаточным условием сходимости метода Якоби является строгое диагональное преобладание в матрице A . При выполнении этого условия метод сходится к единственному решению при любом начальном приближении.

Критерий останова. Итерационный процесс останавливается, когда норма разности между двумя последовательными приближениями становится меньше заданной точности ε

Выполнение работы.

Программа реализует следующие этапы:

Ввод размерности системы N .

Генерация матрицы A с гарантированным диагональным преобладанием. Для каждого элемента вне диагонали генерируется случайное число из диапазона $[-10, 10]$. Диагональный элемент вычисляется как сумма модулей всех внедиагональных элементов плюс большое положительное число, что заведомо обеспечивает строгое преобладание.

Формирование точного решения как случайного вектора. Вычисляется вектор правой части. Таким образом, известно точное решение, с которым впоследствии сравнивается приближённое.

Решение СЛАУ методом Якоби с начальным приближением.

Вывод результатов: число итераций, достигнутая разница между приближениями, полученное решение и его сравнение с точным.

Выводы.

В ходе работы была разработана программа, реализующая метод Якоби для СЛАУ с диагональным преобладанием. Основные результаты:

Метод Якоби успешно сходится для сгенерированных матриц, что подтверждается выполнением достаточного условия сходимости.

Полученное решение совпадает с точным с погрешностью, не превышающей заданного ε .

Количество итераций зависит от размера системы и конкретных значений элементов матрицы, но в тестовых примерах не превысило нескольких сотен, что говорит о быстрой сходимости.

Программа устойчива: при невыполнении сходимости за 10000 итераций выводится сообщение об ошибке.

Предложенный подход (генерация точного решения и правой части) удобен для верификации численных методов, так как позволяет непосредственно сравнивать результат с эталоном.

Таким образом, цель работы достигнута, метод Якоби реализован корректно и пригоден для решения практических задач, где матрица обладает диагональным преобладанием.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <random>

#define MAXIMUM_ELEMENT_VALUE 10
#ifndef EPSILON
#define EPSILON 1e-8
#endif

void Jacobi(int N, double** A, double* F, double* X) {
    double* TempX = new double[N]();
    double* X_prev = new double[N];
    for (int i = 0; i < N; i++) {
        X[i] = 0;
    }
    double diff;
    int iterations = 0;
    const long MAX_ITER = 10000;
    do {
        for (int i = 0; i < N; i++) {
            X_prev[i] = X[i];
        }
        for (int i = 0; i < N; i++) {
            double sum = F[i];
            for (int j = 0; j < N; j++) {
                if (i != j) {
                    sum -= A[i][j] * X_prev[j];
                }
            }
            TempX[i] = sum / A[i][i];
        }
        for (int i = 0; i < N; i++) {
            diff = fabs(TempX[i] - X_prev[i]);
        }
        for (int i = 0; i < N; i++) {
            X[i] = TempX[i];
        }
        iterations++;
        if (iterations >= MAX_ITER) {
            std::cerr << "Превышено максимальное число итераций!\n";
            break;
        }
    } while (diff > EPSILON);
    std::cout << "Сошлось за " << iterations
        << " итераций, |x_prev - x_current| = " << diff << std::endl;
    delete[] TempX;
    delete[] X_prev;
}

double getRandomNumber() {
    static std::random_device rd;
    static std::mt19937 gen(rd());
    static std::uniform_real_distribution<double> dist(
        -MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE
    );
    return dist(gen);
}
```

```

double** createMatrix(int N) {
    double** A = new double*[N];
    for (int i = 0; i < N; ++i) {
        A[i] = new double[N];
        double sum = 0;
        for (int j = 0; j < N; ++j) {
            if (i != j) {
                A[i][j] = getRandomNumber();
                sum += fabs(A[i][j]);
            }
        }
        A[i][i] = sum + 100000000;
    }
    return A;
}

std::pair<double*, double*> getCorrectResponseVectors(double** A, int N) {
    double* X = new double[N];
    double* B = new double[N];
    for (int i = 0; i < N; ++i) {
        X[i] = getRandomNumber();
    }
    for (int i = 0; i < N; ++i) {
        B[i] = 0;
        for (int j = 0; j < N; ++j) {
            B[i] += A[i][j] * X[j];
        }
    }
    return {X, B};
}

void deleteMatrix(double** A, int N) {
    for (int i = 0; i < N; ++i) {
        delete[] A[i];
    }
    delete[] A;
}

int main() {
    int N;
    std::cin >> N;
    auto A = createMatrix(N);
    auto vectors = getCorrectResponseVectors(A, N);
    double* JacobiAnswer = new double[N]();
    Jacobi(N, A, vectors.second, JacobiAnswer);
    std::cout << "Решение методом Якоби:\n";
    for (int i = 0; i < N; i++) {
        std::cout << "X[" << i << "] = " << JacobiAnswer[i]
                  << " (должно быть " << vectors.first[i] << ")\n";
    }
    delete[] vectors.first;
    delete[] vectors.second;
    delete[] JacobiAnswer;
    deleteMatrix(A, N);
    return EXIT_SUCCESS;
}

```